

TrustUs

Server — Production Reference & Hardening Spec

FastAPI + PostgreSQL + Redis — grounded in the real codebase

Documents what is actually built, and what production scale requires next.

Table of Contents

0. Grounding Note

This document reflects the real, already-implemented server (Trustus-server/) inspected directly from your upload — not a re-statement of earlier idealized design documents from before the build started. Where the real implementation differs from earlier plans, this document says so explicitly and documents what actually exists. Where the code is already production-quality, this document says that too — an honest assessment is more useful than a generic checklist.

1. What's Already Genuinely Solid

Worth stating plainly before the gap list: this is not a rough prototype. Several things are already implemented the way a production system should be, not the way “vibe-coded” software often ends up:

- Structured logging (structlog) configured before anything else in the app factory, with request-ID correlation middleware.
- Config validation that hard-fails startup if JWT_SECRET is left at its placeholder value — prevents the single most common real-world security mistake (shipping a default secret).
- OTP hashing via HMAC-SHA256 with constant-time comparison (hmac.compare_digest) — correctly resists timing attacks.
- Refresh tokens are stored server-side only as a SHA-256 hash (in the sessions table), not the raw token — correct practice.
- The public GET /verify/{unit_id} endpoint already has: per-IP rate limiting (30/min), Redis response caching (30s TTL), and a generic 404 on not-found that reveals nothing about whether an ID format is wrong versus simply unregistered — this is exactly the enumeration-resistant design a public anti-fraud endpoint needs.
- File storage is already abstracted behind a local/S3 backend switch, not hardcoded to disk — ready for real deployment without a rewrite.
- SMS provider is abstracted (dummy/msg91/twilio) — swappable without touching business logic, matching the design principle from earlier planning documents.
- Idempotent event submission and single-flight token refresh (client-side) are both implemented as designed.

2. Actual Implemented API Surface

Extracted directly from the route decorators in every app/api/v1/*.py file — this is the real, current contract. 39 endpoints across 10 route files.

File	Routes
auth.py	POST /auth/otp/request · POST /auth/otp/verify · POST /auth/refresh · POST /auth/logout
orgs.py	POST /orgs · GET /orgs/me · POST /orgs/{org_id}/kyc-documents · GET/PATCH /orgs/me/notification-settings · GET /orgs/me/users · POST /orgs/me/users/invite · PATCH /orgs/me/users/{user_id}
network.py	POST /network/invites · POST /network/invites/{invite_code}/accept · GET /network/tree
products.py	POST/GET /products · GET /batches/lookup · POST /batches · GET /batches/{batch_id} · POST /batches/{batch_id}/units · POST /batches/{batch_id}/recall · POST /batches/{batch_id}/export-labels
units.py	GET /units/{unit_id} · GET /units/{unit_id}/history · GET /verify/{unit_id} (public)
containers.py	POST /containers · GET /containers/{container_id} · POST /containers/{container_id}/units · POST /containers/{container_id}/disaggregate

events.py	POST /events · GET /events
alerts.py	GET /alerts · GET/PATCH /alerts/{alert_id} · GET /admin/alerts
social.py	POST/GET /social-listings · GET /social-listings/{listing_id} · DELETE /social-listings/{listing_id}
admin.py	GET /admin/orgs · POST /admin/orgs/{org_id}/approve · GET /admin/fraud-patterns · POST /admin/regulator-access
health.py	GET /health

Note: POST /containers/{container_id}/units is the real route name for adding units to an existing container — earlier design documents referred to this differently; treat this file as the authoritative naming from here forward.

3. Real State Machine (as coded, core/state_machine.py)

This differs from earlier design documents and is simpler and slightly different in shape — document what's real, not the earlier draft.

Current State	event_type	Resulting State
ISSUED / RECEIVED / RETURNED	DISPATCH	IN_TRANSIT
IN_TRANSIT	RECEIVE	RECEIVED
RECEIVED	SALE	SOLD
SOLD	RETURN	RETURNED
any non-flagged state	FLAG (admin/detection only)	FLAGGED
FLAGGED	UNFLAG (admin only)	ISSUED
FLAGGED	anything else	invalid — frozen until UNFLAG

Two things worth confirming deliberately rather than assuming: (1) RECEIVED is a single generic state, not separated into WITH_DISTRIBUTOR / WITH_RETAILER as earlier planning documents proposed — the distinction lives in actor_org_id and the network edge, not the state name itself; (2) a RETURN/RETURNED flow exists that earlier documents did not plan for — confirm the product requirement this serves (e.g. retailer returning unsold stock to a distributor) is intentional and fully specified for the frontend (Section 4 of the Web Dashboard doc should reflect RETURN as an available action wherever SALE currently is, if that's the intended UX).

4. Real Detection Engine (as coded, core/detection.py)

Rule	Threshold (as coded)	Notes
impossible_travel	> 500 km apart in < 1 hour	Config-overridable via IMPOSSIBLE_TRAVEL_KMH_THRESHOLD in config.py, default 150 — confirm which of these two figures (the hardcoded 500km/1hr in detection.py vs. the configurable 150km/h setting) is actually the one wired in; this is worth a direct code cross-check before shipping, since a docstring and a config default disagreeing is a real, findable bug

		class.
unauthorized_route	actor_in_authorized_chain flag on DetectionContext	Confirms the rule exists; the exact severity split (unauthorized-but-registered vs. no edge at all) described in earlier planning documents should be verified against the actual rule body beyond what this review had time to trace line-by-line — flagged as a verify-before-launch item.
excessive_scan_freq	> 10 scans in a 10-minute window	Uses a Redis counter for scan_count_in_window — confirm the counter key includes unit_id (not just org) so frequency is tracked per-unit, matching the rule's intent.

5. Real Public Verify Behavior

Important correction to earlier design assumptions elsewhere in this project: GET /verify/{unit_id} returns exactly three states — genuine, flagged, recalled — not a four-state version including “genuine_unauthorized_channel” that earlier documents proposed. The real implementation also returns the last 3 custody events for chain-of-custody transparency directly in the verify response, which is a genuinely good design decision not present in earlier plans — keep it, and make sure the Web and Mobile front ends actually render those 3 events, not just the status.

6. Auth & Session Architecture (as coded)

- Access tokens: JWT, 15-minute TTL (config default), HS256, carries sub/org_id/role/type.
- Refresh tokens: JWT, 30-day TTL (config default), includes a jti for uniqueness, hashed with SHA-256 before storage in the sessions table for revocation lookup.
- OTP: 6-digit, generated via secrets.randbelow (cryptographically secure), hashed with HMAC-SHA256 keyed on JWT_SECRET.
- Permission chain: get_current_user (decodes + loads User) → require_role(...) → require_active_org — confirmed present in core/permissions.py.

7. Confirmed Gaps — What's Missing for Full Production Readiness

Gap	Why It Matters	Recommendation
No MFA implementation found	Earlier planning documents specified MFA as mandatory for org_admin and platform_admin — no TOTP code path found in security.py or permissions.py.	Add TOTP (e.g. pyotp) enrollment + verification step in the login flow for these two roles before production launch.
No org-rejection endpoint	Admin can approve (POST /admin/orgs/{id}/approve) but there is no corresponding reject-with-reason route — the Admin Guide's org lifecycle assumed one exists.	Add POST /admin/orgs/{org_id}/reject accepting a required reason string, setting approval_status='rejected' and notifying the org.
No regulator-access list/revoke endpoint	POST /admin/regulator-access creates a grant, but there's no GET to list active grants or endpoint to revoke one early.	Add GET /admin/regulator-access and POST /admin/regulator-access/{id}/revoke

		— needed for the Web Dashboard's Regulator Access page to be a complete feature, not just a one-way form.
Background worker present but not verified wired for detection	Dockerfile.worker exists, implying a Celery (or similar) worker process, but this review did not trace whether POST /events actually enqueues detection asynchronously versus running rules inline.	Confirm detection runs off the request path (async worker) before load testing — running fraud rules synchronously inside the event-submission request would directly limit throughput under concurrent load.
No visible connection pool tuning	AsyncSession/engine setup wasn't reviewed for explicit pool_size / max_overflow configuration.	Set explicit SQLAlchemy pool_size and max_overflow based on expected concurrent request volume and your Postgres max_connections — the asyncpg default pool is not sized for production concurrency out of the box.
No visible CORS configuration reviewed in depth	main.py imports CORSMiddleware; the actual allowed-origins list wasn't confirmed.	Ensure allowed origins are an explicit allowlist (your web/mobile app domains) in production, never a wildcard, especially since cookies/bearer tokens are involved.
No load testing evidence	Rate limiting and caching exist for the verify endpoint specifically, but no evidence of load testing for POST /events under concurrent multi-actor scan bursts (e.g. a large container aggregation).	Load test POST /events and the container aggregation endpoints specifically — these are the highest-write-volume paths in real usage, not the read-heavy verify endpoint alone.
/docs disable-in-production intent stated but not confirmed active	main.py's module docstring says “Disables /docs in production” — confirm this is actually conditional on settings.is_production and not just aspirational documentation.	Verify directly; an exposed OpenAPI schema in production is a common, easily-avoided information-disclosure issue.

8. Scaling for N Concurrent Requests — Concrete Recommendations

8.1 Application Tier

- Run multiple Uvicorn/Gunicorn worker processes behind a load balancer — FastAPI's async model handles concurrency well within one process, but CPU-bound work (detection math, JWT signing) still benefits from multiple processes on multi-core hosts.
- Keep the detection engine on the background worker (Celery, per the existing Dockerfile.worker) so a burst of scans never blocks the API's ability to accept the next request.

8.2 Database Tier

- Explicit connection pool sizing (Section 7) sized to (worker processes × pool_size) comfortably under Postgres's max_connections.
- Composite index on events(target_id, timestamp) if not already present — both the history endpoint and detection rules query by this pattern.
- Consider a read replica for GET-heavy endpoints (network tree, alerts list, fraud patterns) once write volume on the primary becomes a bottleneck — not needed at pilot scale, worth planning for before a second-domain expansion.

8.3 Caching & Rate Limiting

- The verify endpoint's existing Redis cache + rate limit pattern (Section 1) should be replicated for any other high-traffic public or semi-public read (e.g. a future public batch-recall-status check).
- Rate limit OTP request/verify per phone number (not just IP) to prevent SMS-cost abuse — confirm this exists; it wasn't directly traced in this review.

8.4 Observability

- Structured logging is already in place (Section 1) — extend it with an APM/tracing tool (e.g. OpenTelemetry) before production launch so slow endpoints under real load are diagnosable, not just logged.
- Add uptime/error-rate alerting specifically on GET /verify and POST /events — these are respectively the most externally visible and the most business-critical write path.

8.5 Backup & Disaster Recovery

- Automated PostgreSQL backups with a tested restore procedure — given the events table is the append-only trust foundation of the entire platform, its durability guarantee matters more than any other component's.

9. Deployment Reference (as coded)

- Dockerfile.api and Dockerfile.worker exist separately — confirms the intended architecture already separates the request-serving process from the background worker, which is the correct shape for Section 8's scaling recommendations.
- Alembic migrations present with an initial schema revision — confirm CI/CD runs migrations automatically on deploy, never manually against production.
- Environment variables are centrally validated via pydantic-settings (config.py) with sensible defaults for optional values and hard failures for required secrets — keep this pattern for any new config added.